

Week2 –Day 3

LangGraph — Stateful, Multi-Step & Cyclical Agent Workflows

Overall Scenario

Real-world AI agents generally perform multiple steps rather than simply receiving an input and producing an output. An agent may need to plan a task, retrieve information, generate a response, evaluate its own result, revise the response, and sometimes ask a human for approval before performing a sensitive or risky action.

For this project, LangGraph was used to design a stateful agent workflow. LangGraph represents an agent as a graph consisting of nodes, edges, conditional transitions, shared state, human interrupts, and persistence mechanisms. The workflow was progressively developed from a simple conceptual graph into a more advanced system containing linear processing, conditional routing, self-correction, human-in-the-loop approval, persistence, and debugging.

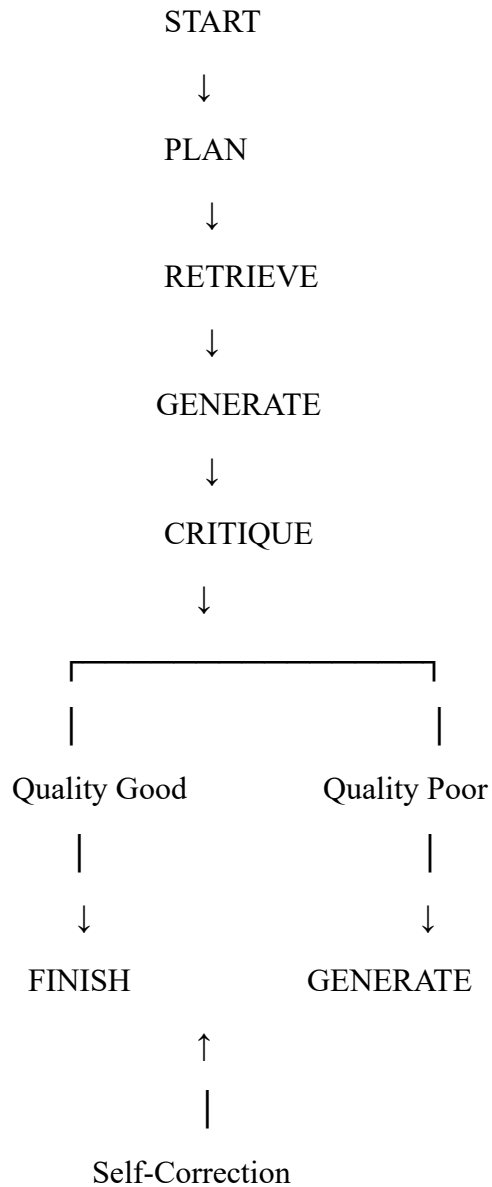
Task 1 — Graph Concepts & State Design

Scenario

The first task focused on understanding how LangGraph represents an agent workflow as a graph.

A research-style assistant workflow was considered in which an agent receives a request, processes it through several stages, evaluates the result, and revises it when necessary.

The basic conceptual workflow was:



Core LangGraph Concepts

StateGraph

StateGraph is the main structure used to construct the workflow. It defines the state schema, nodes, and transitions between nodes.

Nodes

Nodes represent individual operations or stages of the workflow.

Examples include:

- Planning

- Retrieval
- Generation
- Critique
- Human approval
- Finalization

Edges

Edges define the normal transition from one node to another.

For example:

PLAN → RETRIEVE

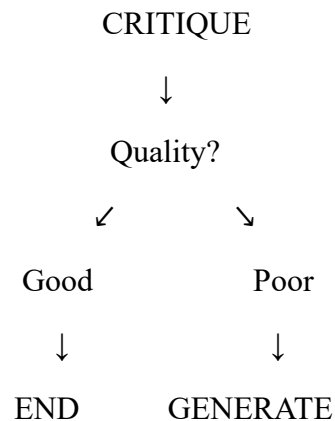
RETRIEVE → GENERATE

GENERATE → CRITIQUE

Conditional Edges

Conditional edges allow the graph to make decisions based on the current state.

For example:



This makes branching and loops explicit.

Shared State

The state is shared between nodes. Each node can read information from the state and update it.

A conceptual state schema contained information such as:

request

draft

critique

quality

retry_count

This state allowed information to move through the complete workflow.

Task 1 Result

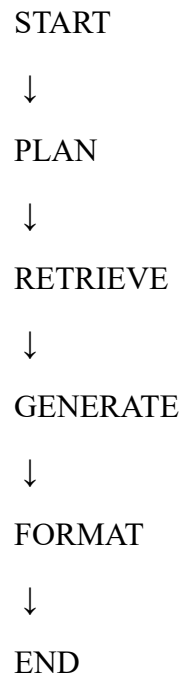
The main outcome of Task 1 was understanding that LangGraph does not treat an agent as one large function. Instead, the agent is divided into controlled stages connected through a shared state.

Task 2 — Build a Linear Graph

Scenario

After understanding the graph concepts, a simple linear workflow was implemented.

The selected workflow was:



The purpose was to understand how information moves from one node to the next.

Workflow

1. Plan

The first node receives the user's request and determines what needs to be done.

2. Retrieve

The retrieval stage represents obtaining the information required to answer the request. The tools from the previous work were used where relevant.

3. Generate

The generated information was converted into a response.

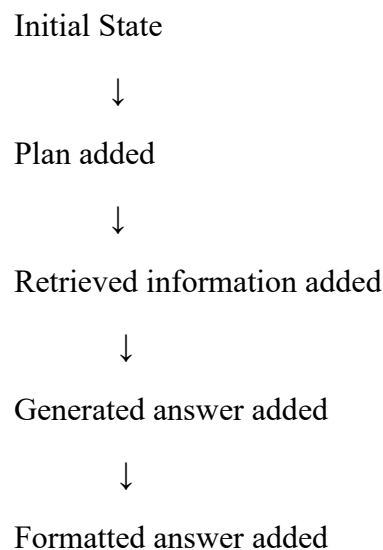
4. Format

The final node formatted the generated response into a clean output.

State Updates

The graph used a shared state object. Each node updated a particular part of the state.

Conceptually:



The state was printed after the processing stages to verify that the values were actually being updated.

Task 2 Result

The linear graph demonstrated the basic execution model of LangGraph:

Input → Node → State Update → Next Node → State Update → Output

This established the foundation required for the more advanced conditional workflow in Task 3.

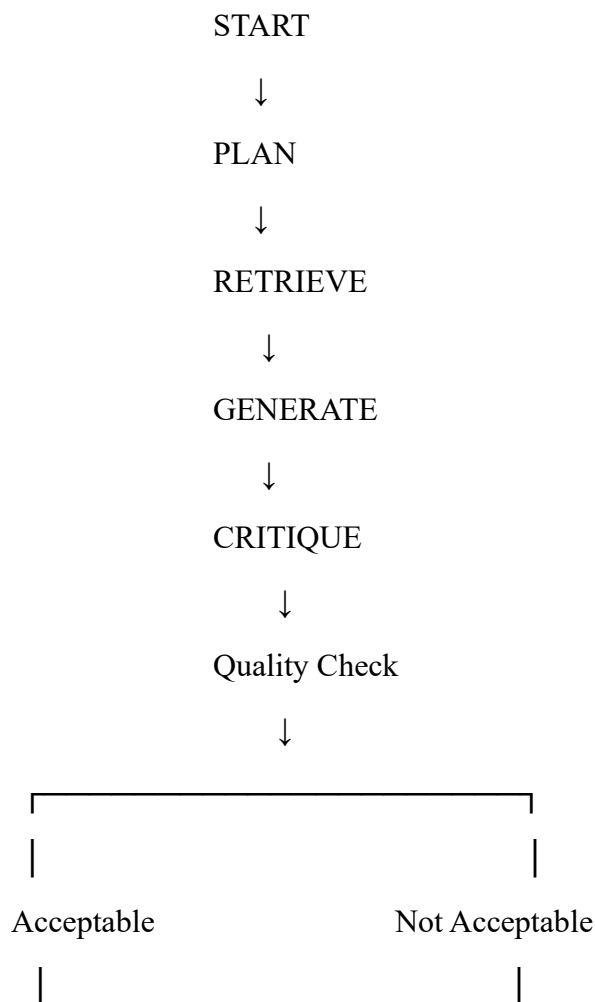
Task 3 — Conditional Edges & Self-Correction Cycles

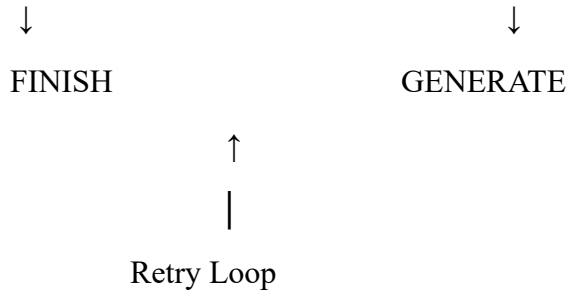
Scenario

A real AI system can sometimes generate an answer that is incomplete, inaccurate, or below the required quality level. Instead of immediately returning that answer, the agent should be able to evaluate it and try again.

For this purpose, a **critique and self-correction loop** was introduced.

Final Workflow



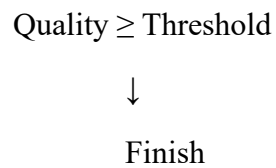


Critique Node

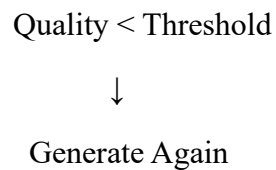
The critique stage evaluated the generated answer.

The workflow checked whether the result met the required quality level.

For example:



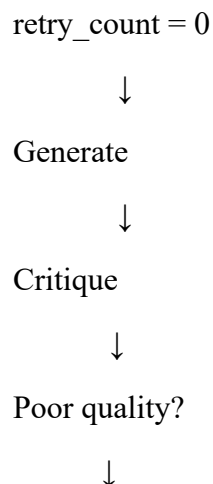
If the quality was below the threshold:



Retry Counter

A retry counter was added to prevent the graph from entering an infinite loop.

Conceptually:



```
retry_count += 1
```



Generate again

After the maximum allowed retries, the workflow stops even if the quality is still below the target.

Why This Is Useful

This creates a **self-correcting agent**.

Instead of:

Generate → Finish

the agent can perform:

Generate → Critique → Improve → Generate → Critique → Finish

Why LangGraph Is Better for This Pattern

A loop like this is difficult to express cleanly with a simple linear AgentExecutor because the workflow needs explicit state, branching, retry limits, and controlled transitions. LangGraph naturally represents these behaviors through nodes, conditional edges, and cycles.

Task 3 Result

Task 3 transformed the linear graph into a more realistic agent workflow capable of evaluation, branching, retrying, and self-correction.

Task 4 — Human-in-the-Loop & Interrupts

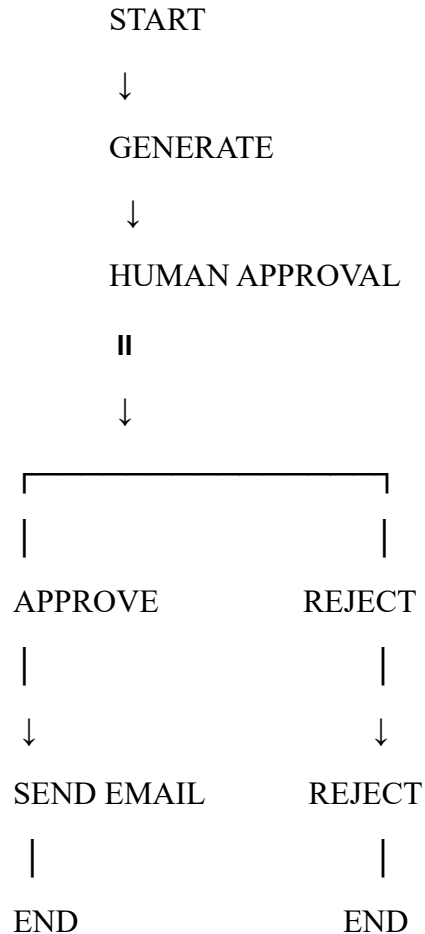
Scenario

Not every action should be performed autonomously by an AI agent.

Some actions can be risky, irreversible, expensive, or externally visible. Therefore, the workflow was extended with a human approval stage.

A simulated email-sending action was selected as the risky operation.

Workflow



Generate Node

The agent first generated an email draft based on the user's request.

For example, the request could be:

Write an email to my manager requesting one day leave.

The agent generated a professional email draft.

Human Approval Node

Before the simulated email was sent, the graph reached the human approval node.

At this point, the workflow used LangGraph's:

`interrupt()`

The graph paused and waited for a human decision.

This demonstrated that the agent does not necessarily have to continue automatically.

Approval Path

If the human responded:

approve

the graph continued:

HUMAN APPROVAL



APPROVED



SEND EMAIL



END

The email action was simulated rather than actually sending an email.

Rejection Path

If the human responded:

reject

the graph followed another path:

HUMAN APPROVAL



REJECTED



STOP

The email was not sent.

Why Human-in-the-Loop Matters

Human approval is particularly important for:

- Financial transactions
- Sending important external communications
- Purchases

- Destructive operations
- Security-sensitive actions
- Legal or business decisions
- Actions that cannot easily be reversed

For low-risk and easily reversible tasks, full autonomy can be appropriate.

Task 4 Result

Task 4 demonstrated a complete human-in-the-loop workflow where the agent pauses before a risky operation and resumes according to human approval or rejection.

Task 5 — Persistence & Debugging

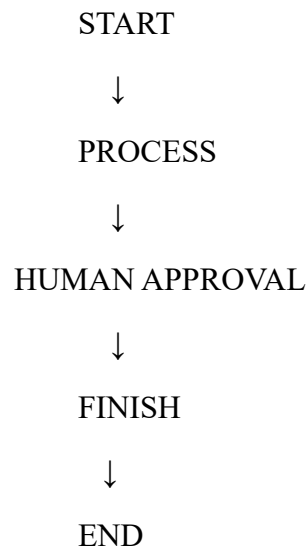
Scenario

The final task focused on making the LangGraph workflow persistent and easier to debug.

A real agent may not finish its work in a single execution. For example, a workflow could pause for human approval and continue later.

Therefore, the graph was configured with a checkpoint.

Workflow

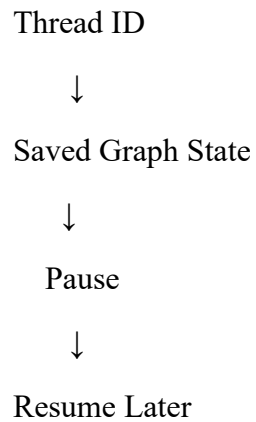


MemorySaver

MemorySaver was used as the checkpointer.

The graph was compiled with the checkpointer so that the state associated with a particular thread_id could be preserved.

Conceptually:



Thread ID

A specific thread ID was assigned to the workflow.

For example:

task5-session-1

The thread ID identifies the particular conversation/workflow state.

This allows the system to retrieve the previous state instead of starting from the beginning.

Persistence Demonstration

The graph was executed and paused at human approval.

The current state was then retrieved using the graph's state functionality.

The saved state contained information such as:

user_input

response

status

approved

This proved that the workflow state remained available after the interrupt.

Resuming the Workflow

The paused workflow was resumed using:

`Command(resume="approve")`

The graph then continued from the interruption point rather than restarting the entire workflow.

State History

The graph's state history was also inspected.

This allowed previous checkpoints to be examined.

The history contained information about:

- Previous states
- State values
- Next nodes
- Configuration
- Execution checkpoints

This is useful for understanding what happened during an agent run.

Debugging

Instead of treating the agent as a black box, the state history allowed the workflow to be inspected step-by-step.

This makes it easier to identify where an agent produced an unexpected result or where a workflow stopped.

Multiple Sessions

A second thread ID was also demonstrated.

For example:

task5-session-1

task5-session-2

Each thread maintains its own workflow state.

Therefore:

Session 1 → State 1

Session 2 → State 2

This demonstrates how LangGraph can manage multiple independent conversations or